

2 Naturalness of Code

April 28, 2026

1 The Naturalness of Code: Analyzing Code at Token Level

In the last lecture we considered two very basic analyses (counting lines of code, and detecting code clones) at character level, by splitting lines. Since our clone analysis looks at lines, it can be very easily fooled simply by adding spurious whitespace (e.g. lines breaks). For example, here is our example function from the last lecture.

```
[1]: code1 = """
public class Foo {
    public void foo(int x) {
        System.out.println("Hello Clone!");
        int j = 10;
        for(int i = 0; i < x; i++) {
            System.out.println("Another iteration");
        }
    }
}
```

Here is a method in a different class that contains exactly the same code, but has some changes to whitespace.

```
[2]: code2 = """
public class Bar {
    public void bar(int x) {
        System.out.
            println("Hello Clone!");
        int j=10;
        for(int i = 0;
            i < x;
            i++) {
            System.out.println("Another iteration");
        }
    }
}
```

Let's have a look what our clone analysis tells us about these two files. For this we need to reproduce the functions we used last time. The first function splits the source code into lines, but ignores

empty lines, lines that contain only braces, or comment lines.

```
[3]: def get_lines(code):
      lines = [l.replace("}", "").replace("{", "").strip() for l in code.
      ↪split("\n")]
      code_lines = [l for l in lines if l and not l.startswith("//")]

      return code_lines
```

The resulting lines are compared directly.

```
[4]: def compare_lines(lines1, lines2):
      matrix = []

      for line1 in lines1:
          row = []
          for line2 in lines2:
              row.append(1 if line1 == line2 else 0)

          matrix.append(row)

      return matrix
```

A clone is found if there are diagonals of 1s in the matrix produced by `compare_lines`. We can get the length of such a diagonal for a given location as follows.

```
[5]: def get_block_at(matrix, x, y):
      block = []

      while (x < len(matrix) and y < len(matrix[x]) and matrix[x][y]):
          block.append((x, y))
          x += 1
          y += 1

      return block
```

To get all diagonals of a minimum size we used the following function.

```
[6]: def get_blocks(matrix, min_size = 3):
      blocks = []
      covered = set()

      width = len(matrix)
      height = len(matrix[0])

      for x in range(width):
          for y in range(height):
              if (x, y) in covered:
                  continue
```

```

        block = get_block_at(matrix, x, y)
        if len(block) >= min_size:
            blocks.append(block)
            for (bx, by) in block:
                covered.add((bx, by))

    return blocks

```

Finally, here is the output function that shows us our clones.

```

[7]: def print_clones(code1, code2):
    lines1 = get_lines(code1)
    lines2 = get_lines(code2)

    matrix = compare_lines(lines1, lines2)
    clones = get_blocks(matrix)

    for clone in clones:
        print("Code in snippet 1:")
        for i, j in clone:
            print(str(i + 1).rjust(3, ' '), ':', lines1[i])

        print("Code in snippet 2:")
        for i, j in clone:
            print(str(j + 1).rjust(3, ' '), ':', lines2[j])
        print("\n")

```

Can a clone be found by comparing code1 and code2?

```

[8]: print_clones(code1, code2)

```

As expected, no clones were found. Although our `get_lines` function removes whitespace at the beginning and the end of lines, it does not look at whitespace within lines. One idea to improve our clone analysis would therefore be to not look at entire lines, but at *words* that are separated by whitespaces.

1.1 Splitting source code into words

```

[9]: code1.split()

```

```

[9]: ['public',
      'class',
      'Foo',
      '{',
      'public',
      'void',
      'foo(int',
      'x)',

```

```
{',
'System.out.println("Hello',
'Clone!");',
'int',
'j',
'=',
'10;',
'for(int',
'i',
'=',
'0;',
'i',
'<',
'x;',
'i++)',
{'',
'System.out.println("Another',
'iteration");',
'}',
'}',
'}']
```

We can easily adapt our clone analysis from using lines to the words produced by the `split` function.

```
[10]: def print_clones(code1, code2):
        lines1 = code1.split()
        lines2 = code2.split()

        matrix = compare_lines(lines1, lines2)
        clones = get_blocks(matrix)

        for clone in clones:
            print("Code in snippet 1:")
            for i, j in clone:
                print(str(i + 1).rjust(3, ' '), ': ', lines1[i])

            print("Code in snippet 2:")
            for i, j in clone:
                print(str(j + 1).rjust(3, ' '), ': ', lines2[j])
            print("\n")
```

Any luck?

```
[11]: print_clones(code1, code2)
```

Code in snippet 1:

```
4 : {
5 : public
6 : void
```

Code in snippet 2:

```
4 : {  
5 : public  
6 : void
```

Code in snippet 1:

```
16 : for(int  
17 : i  
18 : =  
19 : 0;  
20 : i  
21 : <  
22 : x;  
23 : i++)  
24 : {  
25 : System.out.println("Another  
26 : iteration");  
27 : }  
28 : }  
29 : }
```

Code in snippet 2:

```
15 : for(int  
16 : i  
17 : =  
18 : 0;  
19 : i  
20 : <  
21 : x;  
22 : i++)  
23 : {  
24 : System.out.println("Another  
25 : iteration");  
26 : }  
27 : }  
28 : }
```

It found something! However, the first clone is not really interesting, it's just because our minimum size of 3 probably is too low when looking at words rather than lines. The second clone is more interesting: the entire `for`-loop is now detected as a clone, which indeed it is. However, the two lines preceding the loop are not included. The reason is that natural text is separated into words with white spaces, but source code isn't (only). There are also special syntactical variants such as braces etc. In our example, `System.out.println` is not split into multiple words, even though it has multiple components from the point of view of a compiler reading the source code. Similarly, `int j=10` should be more than two words (`int`, `j=10`) – ideally, the same number of words as `int j = 10` (`int`, `j`, `=`, `10`).

There's another problem. Recall that *type 2* clones may differ in terms of literals or identifiers and should still be considered as code clones:

```
[12]: code3 = """
public class Bar {
    public void bar(int x) {
        System.out.println("Completely different text!");
        int j = 200; // completely different numbers
        for(int i = 100; i < x; i++) {
            System.out.println("More completely different text");
        }
    }
}
"""
```

This snippet is identical to the first snippet, except for variable names and literals. However, the clones we can find are not particularly interesting.

```
[13]: print_clones(code1, code3)
```

Code in snippet 1:

```
4 : {
5 : public
6 : void
```

Code in snippet 2:

```
4 : {
5 : public
6 : void
```

Code in snippet 1:

```
12 : int
13 : j
14 : =
```

Code in snippet 2:

```
13 : int
14 : j
15 : =
```

Code in snippet 1:

```
16 : for(int
17 : i
18 : =
```

Code in snippet 2:

```
21 : for(int
22 : i
23 : =
```

Code in snippet 1:

```
20 : i
21 : <
22 : x;
23 : i++)
24 : {
```

Code in snippet 2:

```
25 : i
26 : <
27 : x;
28 : i++)
29 : {
```

Code in snippet 1:

```
27 : }
28 : }
29 : }
```

Code in snippet 2:

```
34 : }
35 : }
36 : }
```

Although there are multiple clones, these just make us wish we had set `min_size` to something much larger than 3, because none of these clones is interesting.

To identify type 2 clones we would need to modify our clone analysis such that it compares all parts of the program except the identifiers and literals. But how can our analysis know what are variables and literals, and how can we get around the problem that words are not always separated by whitespace?

1.2 Lexing Source Code

Source code is processed by a compiler to create an internal tree-representation that allows it to translate it to another language (e.g. assembly), or to interpret it directly. The analysis phase of a compiler consists of two parts: A low-level part called a lexical analyser (mathematically, a finite automaton based on a regular grammar), and a high-level part called a syntax analyser, or parser (mathematically, a push-down automaton based on a context-free grammar, or BNF). Today, we will consider the first part, the lexical analysis.

A lexer identifies substrings of the source program that belong together; these substrings are called *lexemes*.

For example, given the string `for(int i = 0; i < x; i++) {` we would like to build a lexer that outputs the following lexemes: `- for - ( - int - i - = - 0 - ; - i - < - x - ; - i - ++ - ) - {`

Some of the following examples are based on <https://medium.com/@pythonmembers.club/building->

We will start by producing lexemes that separate strings on whitespaces. A simple way to do this would be to simply iterate over a string and store a lexeme whenever we encounter whitespace:

```
[14]: string = 'I love software analysis'
white_space = ' '
lexemes = []

lexeme = ''
for i,char in enumerate(string):
    lexeme += char
    if (i+1 < len(string)):
        if string[i+1] == white_space:
            lexemes.append(lexeme)
            lexeme = ''

lexemes
```

```
[14]: ['I', ' love', ' software']
```

One issue here is that our string does not end in whitespace, so we need to always add the final lexeme:

```
[15]: string = 'I love software analysis'
white_space = ' '
lexemes = []

lexeme = ''
for i,char in enumerate(string):
    lexeme += char
    if (i+1 < len(string)):
        if string[i+1] == white_space:
            lexemes.append(lexeme)
            lexeme = ''

if lexeme:
    lexemes.append(lexeme)

lexemes
```

```
[15]: ['I', ' love', ' software', ' analysis']
```

We are still including the whitespace in our lexemes, which we should avoid really.

```
[16]: string = 'I love software analysis'
white_space = ' '
lexemes = []
```

```

lexeme = ''
for i, char in enumerate(string):
    if char != white_space:
        lexeme += char
    if (i+1 < len(string)):
        if string[i+1] == white_space:
            lexemes.append(lexeme)
            lexeme = ''

if lexeme:
    lexemes.append(lexeme)

lexemes

```

[16]: ['I', 'love', 'software', 'analysis']

We've thus covered lexemes separated by whitespace, but not those separated by syntactical structures of source code. What we need is to define *keywords* that allow our lexer to identify when lexemes represent special syntactical source code elements. Keywords include reserved words like `public`, `class`, but we will treat symbols such as `(` or `{` the same way.

```

[17]: symbols = ['{', '}', '(', ')', '[', ']', '.', '"', '*', '\n', ':', ',', ';',
    ↪ '=', '']

```

```

[18]: keywords = ['public', 'class', 'void', 'main', 'String', 'int', 'for', '++']

```

```

[19]: KEYWORDS = symbols + keywords

```

```

[20]: white_space = [' ', '\t', '\n']

```

```

[21]: lexemes = []
    string = code1

    lexeme = ''
    for i, char in enumerate(string):
        if char not in white_space:
            lexeme += char

        if (i+1 < len(string)):
            if string[i+1] in white_space or string[i+1] in KEYWORDS or lexeme in
    ↪ KEYWORDS:
                if lexeme:
                    lexemes.append(lexeme)
                    lexeme = ''

    if lexeme:
        lexemes.append(lexeme)

```

[22]: lexemes

```
[22]: ['public',  
      'class',  
      'Foo',  
      '{',  
      'public',  
      'void',  
      'foo',  
      '(',  
      'int',  
      'x',  
      ')',  
      '{',  
      'System',  
      '.',  
      'out',  
      '.',  
      'println',  
      '(',  
      '"',  
      'Hello',  
      'Clone!',  
      '"',  
      ')',  
      ';',  
      'int',  
      'j',  
      '=',  
      '10',  
      ';',  
      'for',  
      '(',  
      'int',  
      'i',  
      '=',  
      '0',  
      ';',  
      'i',  
      '<',  
      'x',  
      ';',  
      'i++',  
      ')',  
      '{',  
      'System',  
      '.',
```

```
'out',
'.' ,
'println',
'(',
'\"',
'Another',
'iteration',
'\"',
')',
';',
'}',
'}',
'}']
```

Let's put this in a function.

```
[23]: def tokenize(code):
    lexemes = []
    lexeme = ""
    for i, char in enumerate(code):
        if char not in white_space:
            lexeme += char
        if (i+1 < len(code)):
            if code[i+1] in white_space or code[i+1] in KEYWORDS or lexeme in
KEYWORDS:
                if lexeme:
                    lexemes.append(lexeme)
                    lexeme = ''
        if lexeme:
            lexemes.append(lexeme)
    return lexemes
```

Let's compare the lexemes for our two variants of the same code.

```
[24]: lexemes1 = tokenize(code1)
lexemes2 = tokenize(code2)

for i in range(min(len(lexemes1), len(lexemes2))):
    print(lexemes1[i].ljust(20, ' '), lexemes2[i])
```

public	public
class	class
Foo	Bar
{	{
public	public
void	void
foo	bar
(	(
int	int

x	x
)	)
{	{
System	System
.	.
out	out
.	.
println	println
(	(
"	"
Hello	Hello
Clone!	Clone!
"	"
)	)
;	;
int	int
j	j
=	=
10	10
;	;
for	for
(	(
int	int
i	i
=	=
0	0
;	;
i	i
<	<
x	x
;	;
i++	i++
)	)
{	{
System	System
.	.
out	out
.	.
println	println
(	(
"	"
Another	Another
iteration	iteration
"	"
)	)
;	;
}	}
}	}

```
}                                }
```

This looks promising, so let's adapt our clone detection to use our lexer.

```
[25]: def print_clones(code1, code2):
        lexemes1 = tokenize(code1)
        lexemes2 = tokenize(code2)

        matrix = compare_lines(lexemes1, lexemes2)
        clones = get_blocks(matrix, 20) # more than 3

        for clone in clones:
            print("Code in snippet 1:")
            for i, j in clone:
                print(str(i + 1).rjust(3, ' '), ':', lexemes1[i])

            print("Code in snippet 2:")
            for i, j in clone:
                print(str(j + 1).rjust(3, ' '), ':', lexemes2[j])
            print("\n")
```

```
[26]: print_clones(code1, code2)
```

Code in snippet 1:

```
8 : (
9 : int
10 : x
11 : )
12 : {
13 : System
14 : .
15 : out
16 : .
17 : println
18 : (
19 : "
20 : Hello
21 : Clone!
22 : "
23 : )
24 : ;
25 : int
26 : j
27 : =
28 : 10
29 : ;
30 : for
31 : (
32 : int
```

```

33 : i
34 : =
35 : 0
36 : ;
37 : i
38 : <
39 : x
40 : ;
41 : i++
42 : )
43 : {
44 : System
45 : .
46 : out
47 : .
48 : println
49 : (
50 : "
51 : Another
52 : iteration
53 : "
54 : )
55 : ;
56 : }
57 : }
58 : }
Code in snippet 2:
 8 : (
 9 : int
10 : x
11 : )
12 : {
13 : System
14 : .
15 : out
16 : .
17 : println
18 : (
19 : "
20 : Hello
21 : Clone!
22 : "
23 : )
24 : ;
25 : int
26 : j
27 : =
28 : 10

```

```

29 : ;
30 : for
31 : (
32 : int
33 : i
34 : =
35 : 0
36 : ;
37 : i
38 : <
39 : x
40 : ;
41 : i++
42 : )
43 : {
44 : System
45 : .
46 : out
47 : .
48 : println
49 : (
50 : "
51 : Another
52 : iteration
53 : "
54 : )
55 : ;
56 : }
57 : }
58 : }

```

Our clone detection now matches the entire code of the two variants of the code snippet.

However, let's consider a type 2 clone:

```

[27]: code3 = """
public class Bar {
    public void bar(int x) {
        System.out.println("This is a different string!");
        int j = 50;
        for(int i = 100; i < x; i++) {
            System.out.println("Yet some more different text");
        }
    }
}
"""

```

```
[28]: print_clones(code1, code3)
```

As expected, no code clones were detected because the strings and numbers are different. An obvious way to fix this would be to replace all strings and numbers with some fixed values. However, how do we know which of our lexemes represent strings and numbers?

1.3 From lexemes to tokens

Lexemes match a character pattern, which is associated with a lexical category called a *token*. A token is the name for a set of lexemes, all of which have the same grammatical significance for the parser.

We define a token as a named tuple that tells us the lexeme (its value), the type of token, and its position in the source code.

```
[29]: from collections import namedtuple
Token = namedtuple('Token', ['value', 'type', 'line', 'col'])
```

For our code examples, we might want to distinguish the following token types:

```
[30]: from enum import Enum
class TokenType(Enum):
    INT = 1
    STRING = 2
    KEYWORD = 3
    SYNTAX = 4
    IDENTIFIER = 5
```

The tokenizer needs to distinguish token types based on the characters encountered.

```
[31]: def tokenize(code):
    tokens = []
    lexeme = ""
    line = 0
    col = 0
    i = 0
    while i < len(code):
        char = code[i]
        col += 1
        if char in white_space:
            if char == '\n':
                line += 1
                col = 0
        elif char in KEYWORDS:
            tokens.append(Token(char, TokenType.SYNTAX, line, col))
            lexeme = ''
        else:
            lexeme += char
            while code[i+1] not in KEYWORDS and code[i+1] not in white_space:
                i += 1
```

```

        col += 1
        lexeme += code[i]
    if lexeme in KEYWORDS:
        tokens.append(Token(lexeme, TokenType.KEYWORD, line, col))
    else:
        tokens.append(Token(lexeme, TokenType.IDENTIFIER, line, col))
    lexeme = ''
    i += 1

return tokens

```

```
[32]: tokenize(code1)
```

```

[32]: [Token(value='public', type=<TokenType.KEYWORD: 3>, line=1, col=6),
Token(value='class', type=<TokenType.KEYWORD: 3>, line=1, col=12),
Token(value='Foo', type=<TokenType.IDENTIFIER: 5>, line=1, col=16),
Token(value='{', type=<TokenType.SYNTAX: 4>, line=1, col=18),
Token(value='public', type=<TokenType.KEYWORD: 3>, line=2, col=8),
Token(value='void', type=<TokenType.KEYWORD: 3>, line=2, col=13),
Token(value='foo', type=<TokenType.IDENTIFIER: 5>, line=2, col=17),
Token(value='(', type=<TokenType.SYNTAX: 4>, line=2, col=18),
Token(value='int', type=<TokenType.KEYWORD: 3>, line=2, col=21),
Token(value='x', type=<TokenType.IDENTIFIER: 5>, line=2, col=23),
Token(value=')', type=<TokenType.SYNTAX: 4>, line=2, col=24),
Token(value='{', type=<TokenType.SYNTAX: 4>, line=2, col=26),
Token(value='System', type=<TokenType.IDENTIFIER: 5>, line=3, col=10),
Token(value='.', type=<TokenType.SYNTAX: 4>, line=3, col=11),
Token(value='out', type=<TokenType.IDENTIFIER: 5>, line=3, col=14),
Token(value='.', type=<TokenType.SYNTAX: 4>, line=3, col=15),
Token(value='println', type=<TokenType.IDENTIFIER: 5>, line=3, col=22),
Token(value='(', type=<TokenType.SYNTAX: 4>, line=3, col=23),
Token(value='"', type=<TokenType.SYNTAX: 4>, line=3, col=24),
Token(value='Hello', type=<TokenType.IDENTIFIER: 5>, line=3, col=29),
Token(value='Clone!', type=<TokenType.IDENTIFIER: 5>, line=3, col=36),
Token(value='"', type=<TokenType.SYNTAX: 4>, line=3, col=37),
Token(value=')', type=<TokenType.SYNTAX: 4>, line=3, col=38),
Token(value=';', type=<TokenType.SYNTAX: 4>, line=3, col=39),
Token(value='int', type=<TokenType.KEYWORD: 3>, line=4, col=7),
Token(value='j', type=<TokenType.IDENTIFIER: 5>, line=4, col=9),
Token(value='=', type=<TokenType.SYNTAX: 4>, line=4, col=11),
Token(value='10', type=<TokenType.IDENTIFIER: 5>, line=4, col=14),
Token(value=';', type=<TokenType.SYNTAX: 4>, line=4, col=15),
Token(value='for', type=<TokenType.KEYWORD: 3>, line=5, col=7),
Token(value='(', type=<TokenType.SYNTAX: 4>, line=5, col=8),
Token(value='int', type=<TokenType.KEYWORD: 3>, line=5, col=11),
Token(value='i', type=<TokenType.IDENTIFIER: 5>, line=5, col=13),
Token(value='=', type=<TokenType.SYNTAX: 4>, line=5, col=15),

```

```

Token(value='0', type=<TokenType.IDENTIFIER: 5>, line=5, col=17),
Token(value=';', type=<TokenType.SYNTAX: 4>, line=5, col=18),
Token(value='i', type=<TokenType.IDENTIFIER: 5>, line=5, col=20),
Token(value='<', type=<TokenType.IDENTIFIER: 5>, line=5, col=22),
Token(value='x', type=<TokenType.IDENTIFIER: 5>, line=5, col=24),
Token(value=';', type=<TokenType.SYNTAX: 4>, line=5, col=25),
Token(value='i++', type=<TokenType.IDENTIFIER: 5>, line=5, col=29),
Token(value=')', type=<TokenType.SYNTAX: 4>, line=5, col=30),
Token(value='{', type=<TokenType.SYNTAX: 4>, line=5, col=32),
Token(value='System', type=<TokenType.IDENTIFIER: 5>, line=6, col=12),
Token(value='.', type=<TokenType.SYNTAX: 4>, line=6, col=13),
Token(value='out', type=<TokenType.IDENTIFIER: 5>, line=6, col=16),
Token(value='.', type=<TokenType.SYNTAX: 4>, line=6, col=17),
Token(value='println', type=<TokenType.IDENTIFIER: 5>, line=6, col=24),
Token(value='(', type=<TokenType.SYNTAX: 4>, line=6, col=25),
Token(value='"', type=<TokenType.SYNTAX: 4>, line=6, col=26),
Token(value='Another', type=<TokenType.IDENTIFIER: 5>, line=6, col=33),
Token(value='iteration', type=<TokenType.IDENTIFIER: 5>, line=6, col=43),
Token(value='"', type=<TokenType.SYNTAX: 4>, line=6, col=44),
Token(value=')', type=<TokenType.SYNTAX: 4>, line=6, col=45),
Token(value=';', type=<TokenType.SYNTAX: 4>, line=6, col=46),
Token(value='}', type=<TokenType.SYNTAX: 4>, line=7, col=5),
Token(value='}', type=<TokenType.SYNTAX: 4>, line=8, col=3),
Token(value='}', type=<TokenType.SYNTAX: 4>, line=9, col=1)]

```

We can also identify number tokens if the first character of the lexeme is a digit, string tokens if the first character of a lexeme is a quote, and it is common to skip comments.

```

[33]: def tokenize(code):
    tokens = []
    lexeme = ""
    line = 0
    col = 0
    i = 0

    # TODO: Fix column count

    while i < len(code):
        char = code[i]
        col += 1
        if char == '/':
            if code[i+1] == '/':
                # Skip comments until end
                i += 1
                while code[i] != '\n':
                    i += 1
            elif char.isnumeric():

```

```

        lexeme += char
        while code[i+1].isnumeric():
            i += 1
            char = code[i]
            lexeme += char
        tokens.append(Token(lexeme, TokenType.INT, line, col))
        lexeme = ''
    elif char in white_space:
        if char == '\n':
            line += 1
            col = 0
    elif char == '"':
        while code[i+1] != '"':
            i += 1
            lexeme += code[i]
        i += 1
        tokens.append(Token(lexeme, TokenType.STRING, line, col))
        lexeme = ''
    elif char in KEYWORDS:
        tokens.append(Token(char, TokenType.SYNTAX, line, col))
        lexeme = ''
    else:
        lexeme += char
        while code[i+1] not in KEYWORDS and code[i+1] not in white_space:
            i += 1
            lexeme += code[i]
        if lexeme in KEYWORDS:
            tokens.append(Token(lexeme, TokenType.KEYWORD, line, col))
        else:
            tokens.append(Token(lexeme, TokenType.IDENTIFIER, line, col))
        lexeme = ''
    i += 1

return tokens

```

```
[34]: tokenize(code1)
```

```

[34]: [Token(value='public', type=<TokenType.KEYWORD: 3>, line=1, col=1),
      Token(value='class', type=<TokenType.KEYWORD: 3>, line=1, col=3),
      Token(value='Foo', type=<TokenType.IDENTIFIER: 5>, line=1, col=5),
      Token(value='{', type=<TokenType.SYNTAX: 4>, line=1, col=7),
      Token(value='public', type=<TokenType.KEYWORD: 3>, line=2, col=3),
      Token(value='void', type=<TokenType.KEYWORD: 3>, line=2, col=5),
      Token(value='foo', type=<TokenType.IDENTIFIER: 5>, line=2, col=7),
      Token(value='(', type=<TokenType.SYNTAX: 4>, line=2, col=8),
      Token(value='int', type=<TokenType.KEYWORD: 3>, line=2, col=9),

```

```

Token(value='x', type=<TokenType.IDENTIFIER: 5>, line=2, col=11),
Token(value=')', type=<TokenType.SYNTAX: 4>, line=2, col=12),
Token(value='{', type=<TokenType.SYNTAX: 4>, line=2, col=14),
Token(value='System', type=<TokenType.IDENTIFIER: 5>, line=3, col=5),
Token(value='.', type=<TokenType.SYNTAX: 4>, line=3, col=6),
Token(value='out', type=<TokenType.IDENTIFIER: 5>, line=3, col=7),
Token(value='.', type=<TokenType.SYNTAX: 4>, line=3, col=8),
Token(value='println', type=<TokenType.IDENTIFIER: 5>, line=3, col=9),
Token(value='(', type=<TokenType.SYNTAX: 4>, line=3, col=10),
Token(value='Hello Clone!', type=<TokenType.STRING: 2>, line=3, col=11),
Token(value=')', type=<TokenType.SYNTAX: 4>, line=3, col=12),
Token(value=';', type=<TokenType.SYNTAX: 4>, line=3, col=13),
Token(value='int', type=<TokenType.KEYWORD: 3>, line=4, col=5),
Token(value='j', type=<TokenType.IDENTIFIER: 5>, line=4, col=7),
Token(value='=', type=<TokenType.SYNTAX: 4>, line=4, col=9),
Token(value='10', type=<TokenType.INT: 1>, line=4, col=11),
Token(value=';', type=<TokenType.SYNTAX: 4>, line=4, col=12),
Token(value='for', type=<TokenType.KEYWORD: 3>, line=5, col=5),
Token(value='(', type=<TokenType.SYNTAX: 4>, line=5, col=6),
Token(value='int', type=<TokenType.KEYWORD: 3>, line=5, col=7),
Token(value='i', type=<TokenType.IDENTIFIER: 5>, line=5, col=9),
Token(value='=', type=<TokenType.SYNTAX: 4>, line=5, col=11),
Token(value='0', type=<TokenType.INT: 1>, line=5, col=13),
Token(value=';', type=<TokenType.SYNTAX: 4>, line=5, col=14),
Token(value='i', type=<TokenType.IDENTIFIER: 5>, line=5, col=16),
Token(value='<', type=<TokenType.IDENTIFIER: 5>, line=5, col=18),
Token(value='x', type=<TokenType.IDENTIFIER: 5>, line=5, col=20),
Token(value=';', type=<TokenType.SYNTAX: 4>, line=5, col=21),
Token(value='i++', type=<TokenType.IDENTIFIER: 5>, line=5, col=23),
Token(value=')', type=<TokenType.SYNTAX: 4>, line=5, col=24),
Token(value='{', type=<TokenType.SYNTAX: 4>, line=5, col=26),
Token(value='System', type=<TokenType.IDENTIFIER: 5>, line=6, col=7),
Token(value='.', type=<TokenType.SYNTAX: 4>, line=6, col=8),
Token(value='out', type=<TokenType.IDENTIFIER: 5>, line=6, col=9),
Token(value='.', type=<TokenType.SYNTAX: 4>, line=6, col=10),
Token(value='println', type=<TokenType.IDENTIFIER: 5>, line=6, col=11),
Token(value='(', type=<TokenType.SYNTAX: 4>, line=6, col=12),
Token(value='Another iteration', type=<TokenType.STRING: 2>, line=6, col=13),
Token(value=')', type=<TokenType.SYNTAX: 4>, line=6, col=14),
Token(value=';', type=<TokenType.SYNTAX: 4>, line=6, col=15),
Token(value='}', type=<TokenType.SYNTAX: 4>, line=7, col=5),
Token(value='}', type=<TokenType.SYNTAX: 4>, line=8, col=3),
Token(value='}', type=<TokenType.SYNTAX: 4>, line=9, col=1)]

```

[35]: tokenize(code2)

```

[35]: [Token(value='public', type=<TokenType.KEYWORD: 3>, line=1, col=1),
Token(value='class', type=<TokenType.KEYWORD: 3>, line=1, col=3),
Token(value='Bar', type=<TokenType.IDENTIFIER: 5>, line=1, col=5),
Token(value='{', type=<TokenType.SYNTAX: 4>, line=1, col=7),
Token(value='public', type=<TokenType.KEYWORD: 3>, line=2, col=3),
Token(value='void', type=<TokenType.KEYWORD: 3>, line=2, col=5),
Token(value='bar', type=<TokenType.IDENTIFIER: 5>, line=2, col=7),
Token(value='(', type=<TokenType.SYNTAX: 4>, line=2, col=8),
Token(value='int', type=<TokenType.KEYWORD: 3>, line=2, col=9),
Token(value='x', type=<TokenType.IDENTIFIER: 5>, line=2, col=11),
Token(value=')', type=<TokenType.SYNTAX: 4>, line=2, col=12),
Token(value='{', type=<TokenType.SYNTAX: 4>, line=2, col=14),
Token(value='System', type=<TokenType.IDENTIFIER: 5>, line=3, col=5),
Token(value='.', type=<TokenType.SYNTAX: 4>, line=3, col=6),
Token(value='out', type=<TokenType.IDENTIFIER: 5>, line=3, col=7),
Token(value='.', type=<TokenType.SYNTAX: 4>, line=3, col=8),
Token(value='println', type=<TokenType.IDENTIFIER: 5>, line=4, col=13),
Token(value='(', type=<TokenType.SYNTAX: 4>, line=4, col=14),
Token(value='Hello Clone!', type=<TokenType.STRING: 2>, line=4, col=15),
Token(value=')', type=<TokenType.SYNTAX: 4>, line=4, col=16),
Token(value=';', type=<TokenType.SYNTAX: 4>, line=4, col=17),
Token(value='int', type=<TokenType.KEYWORD: 3>, line=5, col=5),
Token(value='j', type=<TokenType.IDENTIFIER: 5>, line=5, col=7),
Token(value='=', type=<TokenType.SYNTAX: 4>, line=5, col=8),
Token(value='10', type=<TokenType.INT: 1>, line=5, col=9),
Token(value=';', type=<TokenType.SYNTAX: 4>, line=5, col=10),
Token(value='for', type=<TokenType.KEYWORD: 3>, line=6, col=5),
Token(value='(', type=<TokenType.SYNTAX: 4>, line=6, col=6),
Token(value='int', type=<TokenType.KEYWORD: 3>, line=6, col=7),
Token(value='i', type=<TokenType.IDENTIFIER: 5>, line=6, col=9),
Token(value='=', type=<TokenType.SYNTAX: 4>, line=6, col=11),
Token(value='0', type=<TokenType.INT: 1>, line=6, col=13),
Token(value=';', type=<TokenType.SYNTAX: 4>, line=6, col=14),
Token(value='i', type=<TokenType.IDENTIFIER: 5>, line=7, col=9),
Token(value='<', type=<TokenType.IDENTIFIER: 5>, line=7, col=11),
Token(value='x', type=<TokenType.IDENTIFIER: 5>, line=7, col=13),
Token(value=';', type=<TokenType.SYNTAX: 4>, line=7, col=14),
Token(value='i++', type=<TokenType.IDENTIFIER: 5>, line=8, col=9),
Token(value=')', type=<TokenType.SYNTAX: 4>, line=8, col=10),
Token(value='{', type=<TokenType.SYNTAX: 4>, line=8, col=12),
Token(value='System', type=<TokenType.IDENTIFIER: 5>, line=9, col=9),
Token(value='.', type=<TokenType.SYNTAX: 4>, line=9, col=10),
Token(value='out', type=<TokenType.IDENTIFIER: 5>, line=9, col=11),
Token(value='.', type=<TokenType.SYNTAX: 4>, line=9, col=12),
Token(value='println', type=<TokenType.IDENTIFIER: 5>, line=9, col=13),
Token(value='(', type=<TokenType.SYNTAX: 4>, line=9, col=14),
Token(value='Another iteration', type=<TokenType.STRING: 2>, line=9, col=15),

```

```
Token(value='')', type=<TokenType.SYNTAX: 4>, line=9, col=16),
Token(value=';', type=<TokenType.SYNTAX: 4>, line=9, col=17),
Token(value='}', type=<TokenType.SYNTAX: 4>, line=10, col=5),
Token(value='}', type=<TokenType.SYNTAX: 4>, line=11, col=3),
Token(value='}', type=<TokenType.SYNTAX: 4>, line=12, col=1)]
```

Given our new tokenizer, we can now define a function that normalizes strings and numbers by replacing them with a constant placeholder value.

```
[36]: def normalized_tokens(tokens):
    normalized_tokens = []
    for token in tokens:
        if token.type == TokenType.INT:
            normalized_tokens.append(Token("<INT>", TokenType.INT, token.line,
↪token.col))
        elif token.type == TokenType.STRING:
            normalized_tokens.append(Token("<STR>", TokenType.STRING, token.
↪line, token.col))
        else:
            normalized_tokens.append(token)

    return normalized_tokens
```

```
[37]: normalized_tokens(tokenize(code1))
```

```
[37]: [Token(value='public', type=<TokenType.KEYWORD: 3>, line=1, col=1),
Token(value='class', type=<TokenType.KEYWORD: 3>, line=1, col=3),
Token(value='Foo', type=<TokenType.IDENTIFIER: 5>, line=1, col=5),
Token(value='{', type=<TokenType.SYNTAX: 4>, line=1, col=7),
Token(value='public', type=<TokenType.KEYWORD: 3>, line=2, col=3),
Token(value='void', type=<TokenType.KEYWORD: 3>, line=2, col=5),
Token(value='foo', type=<TokenType.IDENTIFIER: 5>, line=2, col=7),
Token(value='(', type=<TokenType.SYNTAX: 4>, line=2, col=8),
Token(value='int', type=<TokenType.KEYWORD: 3>, line=2, col=9),
Token(value='x', type=<TokenType.IDENTIFIER: 5>, line=2, col=11),
Token(value='')', type=<TokenType.SYNTAX: 4>, line=2, col=12),
Token(value='{', type=<TokenType.SYNTAX: 4>, line=2, col=14),
Token(value='System', type=<TokenType.IDENTIFIER: 5>, line=3, col=5),
Token(value='.', type=<TokenType.SYNTAX: 4>, line=3, col=6),
Token(value='out', type=<TokenType.IDENTIFIER: 5>, line=3, col=7),
Token(value='.', type=<TokenType.SYNTAX: 4>, line=3, col=8),
Token(value='println', type=<TokenType.IDENTIFIER: 5>, line=3, col=9),
Token(value='(', type=<TokenType.SYNTAX: 4>, line=3, col=10),
Token(value='<STR>', type=<TokenType.STRING: 2>, line=3, col=11),
Token(value='')', type=<TokenType.SYNTAX: 4>, line=3, col=12),
Token(value=';', type=<TokenType.SYNTAX: 4>, line=3, col=13),
Token(value='int', type=<TokenType.KEYWORD: 3>, line=4, col=5),
```

```

Token(value='j', type=<TokenType.IDENTIFIER: 5>, line=4, col=7),
Token(value='=', type=<TokenType.SYNTAX: 4>, line=4, col=9),
Token(value='<INT>', type=<TokenType.INT: 1>, line=4, col=11),
Token(value=';', type=<TokenType.SYNTAX: 4>, line=4, col=12),
Token(value='for', type=<TokenType.KEYWORD: 3>, line=5, col=5),
Token(value='(', type=<TokenType.SYNTAX: 4>, line=5, col=6),
Token(value='int', type=<TokenType.KEYWORD: 3>, line=5, col=7),
Token(value='i', type=<TokenType.IDENTIFIER: 5>, line=5, col=9),
Token(value='=', type=<TokenType.SYNTAX: 4>, line=5, col=11),
Token(value='<INT>', type=<TokenType.INT: 1>, line=5, col=13),
Token(value=';', type=<TokenType.SYNTAX: 4>, line=5, col=14),
Token(value='i', type=<TokenType.IDENTIFIER: 5>, line=5, col=16),
Token(value='<', type=<TokenType.IDENTIFIER: 5>, line=5, col=18),
Token(value='x', type=<TokenType.IDENTIFIER: 5>, line=5, col=20),
Token(value=';', type=<TokenType.SYNTAX: 4>, line=5, col=21),
Token(value='i++', type=<TokenType.IDENTIFIER: 5>, line=5, col=23),
Token(value=')', type=<TokenType.SYNTAX: 4>, line=5, col=24),
Token(value='{', type=<TokenType.SYNTAX: 4>, line=5, col=26),
Token(value='System', type=<TokenType.IDENTIFIER: 5>, line=6, col=7),
Token(value='.', type=<TokenType.SYNTAX: 4>, line=6, col=8),
Token(value='out', type=<TokenType.IDENTIFIER: 5>, line=6, col=9),
Token(value='.', type=<TokenType.SYNTAX: 4>, line=6, col=10),
Token(value='println', type=<TokenType.IDENTIFIER: 5>, line=6, col=11),
Token(value='(', type=<TokenType.SYNTAX: 4>, line=6, col=12),
Token(value='<STR>', type=<TokenType.STRING: 2>, line=6, col=13),
Token(value=')', type=<TokenType.SYNTAX: 4>, line=6, col=14),
Token(value=';', type=<TokenType.SYNTAX: 4>, line=6, col=15),
Token(value='}', type=<TokenType.SYNTAX: 4>, line=7, col=5),
Token(value='}', type=<TokenType.SYNTAX: 4>, line=8, col=3),
Token(value='}', type=<TokenType.SYNTAX: 4>, line=9, col=1)]

```

To use this in our clone analysis we need to refine our matrix generation to look at the lexemes of the tokens, since the comparison should not consider the location.

```

[38]: def compare_tokens(tokens1, tokens2):
    matrix = []

    for token1 in tokens1:
        row = []
        for token2 in tokens2:
            row.append(1 if token1.value == token2.value else 0)

        matrix.append(row)

    return matrix

```

Finally, here's our refined clone analysis that works at token level. We also refine the analysis to print the affected lines instead of lists of tokens.

```
[39]: def print_clones(code1, code2):
    tokens1 = tokenize(code1)
    tokens2 = tokenize(code2)

    normalized_tokens1 = normalized_tokens(tokens1)
    normalized_tokens2 = normalized_tokens(tokens2)

    matrix = compare_tokens(normalized_tokens1, normalized_tokens2)

    clones = get_blocks(matrix, 20)

    for clone in clones:
        print("Clone")
        lines1 = []
        lines2 = []
        for i, j in clone:
            line = tokens1[i].line
            if line not in lines1:
                lines1.append(line)

            line = tokens2[j].line
            if line not in lines2:
                lines2.append(line)

        print("Code in snippet 1:")
        code_lines = code1.split('\n')
        for line in lines1:
            print(f"{line+1}: {code_lines[line+1]}")

        print("Code in snippet 2:")
        code_lines = code2.split('\n')
        for line in lines2:
            print(f"{line+1}: {code_lines[line+1]}")
        print("\n")
```

First a sanity check: Does it still work on our type 1 clone?

```
[40]: print_clones(code1, code2)
```

```
Clone
Code in snippet 1:
3:   System.out.println("Hello Clone!");
4:   int j = 10;
5:   for(int i = 0; i < x; i++) {
6:       System.out.println("Another iteration");
7:   }
8: }
9: }
```

```

10:
Code in snippet 2:
3:     System.out.
4:         println("Hello Clone!");
5:     int j=10;
6:     for(int i = 0;
7:         i < x;
8:         i++) {
9:         System.out.println("Another iteration");
10:    }
11: }
12: }
13:

```

(Note that our clone detection is taking a number of shortcuts; we could improve how we are analyzing the matrix. If you reduce the `min_size` you'll currently see some redundant code clones.)

Now let's consider our type 2 clone.

```
[41]: print_clones(code1, code3)
```

```

Clone
Code in snippet 1:
3:     System.out.println("Hello Clone!");
4:     int j = 10;
5:     for(int i = 0; i < x; i++) {
6:         System.out.println("Another iteration");
7:     }
8: }
9: }
10:
Code in snippet 2:
3:     System.out.println("This is a different string!");
4:     int j = 50;
5:     for(int i = 100; i < x; i++) {
6:         System.out.println("Yet some more different text");
7:     }
8: }
9: }
10:

```

It works!

In practice, we wouldn't need to create a lexer by hand. Language recognition is an established problem in computer science, and compiler construction a mature topic with many supporting tools. The classical lexer generator tool is [Flex](#), which is based on the classic Unix utility [Lex](#). Tokens

are specified as regular expressions, and Flex automatically generates the code that processes a character stream to generate tokens.

For Python code aiming to tokenize Java code, there is the [javalang](#) parser framework, which provides a tokenizer.

```
[42]: import javalang
```

The output in principle is similar to what our tokenizer does.

```
[43]: list(javalang.tokenizer.tokenize(code1))
```

```
[43]: [Modifier "public" line 2, position 1,
      Keyword "class" line 2, position 8,
      Identifier "Foo" line 2, position 14,
      Separator "{" line 2, position 18,
      Modifier "public" line 3, position 3,
      Keyword "void" line 3, position 10,
      Identifier "foo" line 3, position 15,
      Separator "(" line 3, position 18,
      BasicType "int" line 3, position 19,
      Identifier "x" line 3, position 23,
      Separator ")" line 3, position 24,
      Separator "{" line 3, position 26,
      Identifier "System" line 4, position 5,
      Separator "." line 4, position 11,
      Identifier "out" line 4, position 12,
      Separator "." line 4, position 15,
      Identifier "println" line 4, position 16,
      Separator "(" line 4, position 23,
      String "'Hello Clone!'" line 4, position 24,
      Separator ")" line 4, position 38,
      Separator ";" line 4, position 39,
      BasicType "int" line 5, position 5,
      Identifier "j" line 5, position 9,
      Operator "=" line 5, position 11,
      DecimalInteger "10" line 5, position 13,
      Separator ";" line 5, position 15,
      Keyword "for" line 6, position 5,
      Separator "(" line 6, position 8,
      BasicType "int" line 6, position 9,
      Identifier "i" line 6, position 13,
      Operator "=" line 6, position 15,
      DecimalInteger "0" line 6, position 17,
      Separator ";" line 6, position 18,
      Identifier "i" line 6, position 20,
      Operator "<" line 6, position 22,
      Identifier "x" line 6, position 24,
```

```

Separator ";" line 6, position 25,
Identifier "i" line 6, position 27,
Operator "++" line 6, position 28,
Separator ")" line 6, position 30,
Separator "{" line 6, position 32,
Identifier "System" line 7, position 7,
Separator "." line 7, position 13,
Identifier "out" line 7, position 14,
Separator "." line 7, position 17,
Identifier "println" line 7, position 18,
Separator "(" line 7, position 25,
String "\"Another iteration\"" line 7, position 26,
Separator ")" line 7, position 45,
Separator ";" line 7, position 46,
Separator "}" line 8, position 5,
Separator "}" line 9, position 3,
Separator "}" line 10, position 1]

```

It would be straightforward to adapt out clone detection to use javalang.

1.4 Language Models

The tokenizer allows us to split source code properly into words, just like we are able to do for regular text by whitespaces.

Natural languages like English are rich and powerful, but in practice most human utterances are simple, repetitive and predictable. These utterances can be very usefully modeled using modern statistical methods. This has led to the phenomenal success of Natural Language Processing (NLP), i.e. statistical approaches to speech recognition, natural language translation, question-answering, and text mining and comprehension.

Since we can now split source code into words just like we can do for natural language, this raises the question whether we can apply NLP methods also to source code. Hindle et al. postulated that software is similarly natural, in the sense that it is created by humans at work, with all the attendant constraints and limitations, and is therefore also repetitive and predictable.

Abram Hindle, Earl T. Barr, Zhendong Su, Mark Gabel, and Premkumar Devanbu. On the naturalness of software. In 2012 34th International Conference on Software Engineering (ICSE), pages 837–847, 2012.6

The *Naturalness Hypothesis* states that code can be usefully modeled by statistical language models, and such models can be leveraged to support software engineers.

A language model essentially assigns a probability to an utterance. It is typically formulated in terms of conditional probabilities, where the probability of the next word in a sequence is conditioned on all previous words in the sequence. Let's take a closer look at language models in the scope of natural language processing, before moving on to see how they can be used with software.

1.4.1 n-gram models

The n-gram model is a simple statistical language model. Consider the sequence of tokens in a document (in our case, a system s), $a_1 a_2 \dots a_i \dots a_n$. An n-gram model estimates the probability of a sequence by statistically estimating how likely tokens are to follow other tokens. Thus, we can estimate the probability of a document based on the product of a series of conditional probabilities:

$$p(s) = p(a_1) \times p(a_2|a_1) \times p(a_3|a_1 a_2) \dots p(a_n|a_1 \dots a_{n-1})$$

A n-gram model assumes a Markov property, i.e., token occurrences are influenced only by a limited prefix of length n , thus for 4-gram models, we assume

$$p(a_i|a_1 \dots a_{i-1}) \cong p(a_i|a_{i-3} a_{i-2} a_{i-1})$$

These models are estimated from a corpus using simple maximum-likelihood based frequency-counting of token sequences. Thus, if $*$ is a wildcard, we ask, how relatively often are the tokens a_1, a_2, a_3 followed by a_4 :

$$p(a_4|a_1 a_2 a_3) = \frac{\text{count}(a_1 a_2 a_3 a_4)}{\text{count}(a_1 a_2 a_3 *)}$$

We will use the well-established NLTK library for n-gram models.

```
[44]: from nltk.util import ngrams
```

Let's assume an arbitrary sentence in natural language.

```
[45]: string = "there is a cat licking your birthday cake"
```

Let's set $n=2$ to start with. Using NLTK, we can extract all bigrams from our sentence easily.

```
[46]: n = 2
      list(ngrams(string.split(), n))
```

```
[46]: [('there', 'is'),
      ('is', 'a'),
      ('a', 'cat'),
      ('cat', 'licking'),
      ('licking', 'your'),
      ('your', 'birthday'),
      ('birthday', 'cake')]
```

For common values of n NLTK also offers functions we can directly call without specifying n :

```
[47]: from nltk.util import bigrams
      list(bigrams(string.split()))
```

```
[47]: [('there', 'is'),
      ('is', 'a'),
      ('a', 'cat'),
      ('cat', 'licking'),
      ('licking', 'your'),
      ('your', 'birthday'),
      ('birthday', 'cake')]
```

Note that the first (**there**) and last (**cake**) word only occur once, while all other words are part of two bigrams. In order to allow the model to capture how often sentences start with **there** and end with **cake** NLTK let's us add special padding symbols to the sentence before splitting it into n-grams.

```
[48]: from nltk.lm.preprocessing import pad_both_ends
      list(bigrams(pad_both_ends(string.split(), n=2)))
```

```
[48]: [('<s>', 'there'),
      ('there', 'is'),
      ('is', 'a'),
      ('a', 'cat'),
      ('cat', 'licking'),
      ('licking', 'your'),
      ('your', 'birthday'),
      ('birthday', 'cake'),
      ('cake', '</s>')]
```

To make our model more robust we could also train it on unigrams (single words) as well as bigrams, its main source of information. NLTK once again helpfully provides a function called everygrams.

```
[49]: from nltk.util import everygrams
      list(everygrams(string.split(), max_len=2))
```

```
[49]: [('there',),
      ('there', 'is'),
      ('is',),
      ('is', 'a'),
      ('a',),
      ('a', 'cat'),
      ('cat',),
      ('cat', 'licking'),
      ('licking',),
      ('licking', 'your'),
      ('your',),
      ('your', 'birthday'),
      ('birthday',),
      ('birthday', 'cake'),
      ('cake',)]
```

During training and evaluation our model will rely on a vocabulary that defines which words are “known” to the model. To create this vocabulary we need to pad our sentences (just like for counting ngrams) and then combine the sentences into one flat stream of words. This is done by the pipeline function.

```
[50]: from nltk.lm.preprocessing import padded_everygram_pipeline
      string_tokens = ["there is a cat licking your birthday cake".split(),
                      "he can't read so he does not know that the cake is not for_
                      ↪him".split(),
```

```

        "it might be his birthday too but the chance of that is slim".
↪split()
    ]

train, vocab = padded_everygram_pipeline(2, string_tokens)

```

So as to avoid re-creating the text in memory, both train and vocab are lazy iterators. They are evaluated on demand at training time.

For the sake of understanding the output of padded_everygram_pipeline, we'll “materialize” the lazy iterators by casting them into a list.

```

[51]: training_ngrams, padded_sentences = padded_everygram_pipeline(2, string_tokens)
      for ngramlize_sent in training_ngrams:
          print(list(ngramlize_sent))

```

```

[('<s>',), ('<s>', 'there'), ('there',), ('there', 'is'), ('is',), ('is', 'a'),
('a',), ('a', 'cat'), ('cat',), ('cat', 'licking'), ('licking',), ('licking',
'your'), ('your',), ('your', 'birthday'), ('birthday',), ('birthday', 'cake'),
('cake',), ('cake', '</s>'), ('</s>',)]
[('<s>',), ('<s>', 'he'), ('he',), ('he', "can't"), ("can't",), ("can't",
'read'), ('read',), ('read', 'so'), ('so',), ('so', 'he'), ('he',), ('he',
'does'), ('does',), ('does', 'not'), ('not',), ('not', 'know'), ('know',),
('know', 'that'), ('that',), ('that', 'the'), ('the',), ('the', 'cake'),
('cake',), ('cake', 'is'), ('is',), ('is', 'not'), ('not',), ('not', 'for'),
('for',), ('for', 'him'), ('him',), ('him', '</s>'), ('</s>',)]
[('<s>',), ('<s>', 'it'), ('it',), ('it', 'might'), ('might',), ('might', 'be'),
('be',), ('be', 'his'), ('his',), ('his', 'birthday'), ('birthday',),
('birthday', 'too'), ('too',), ('too', 'but'), ('but',), ('but', 'the'),
('the',), ('the', 'chance'), ('chance',), ('chance', 'of'), ('of',), ('of',
'that'), ('that',), ('that', 'is'), ('is',), ('is', 'slim'), ('slim',), ('slim',
'</s>'), ('</s>',)]

```

```

[52]: list(padded_sentences)

```

```

[52]: ['<s>',
      'there',
      'is',
      'a',
      'cat',
      'licking',
      'your',
      'birthday',
      'cake',
      '</s>',
      '<s>',
      'he',
      "can't",
      'read',

```

```
'so',  
'he',  
'does',  
'not',  
'know',  
'that',  
'the',  
'cake',  
'is',  
'not',  
'for',  
'him',  
'</s>',  
'<s>',  
'it',  
'might',  
'be',  
'his',  
'birthday',  
'too',  
'but',  
'the',  
'chance',  
'of',  
'that',  
'is',  
'slim',  
'</s>']
```

Having prepared our data we are ready to start training a model. As a simple example, let us train a Maximum Likelihood Estimator (MLE).

We only need to specify the highest ngram order to instantiate it.

```
[53]: from nltk.lm import MLE  
      lm = MLE(2)
```

The model initially has no content:

```
[54]: len(lm.vocab)
```

```
[54]: 0
```

We need to train the model with our n-grams.

```
[55]: lm.fit(train, vocab)
```

```
[56]: len(lm.vocab)
```

[56]: 31

We can look up vocabulary in the model, for example to check that our first sentence is contained in the model.

```
[57]: lm.vocab.lookup(string_tokens[0])
```

[57]: ('there', 'is', 'a', 'cat', 'licking', 'your', 'birthday', 'cake')

If we lookup the vocab on unseen sentences not from the training data, NLTK automatically replaces words not in the vocabulary with <UNK>.

```
[58]: lm.vocab.lookup('there is a cat licking your birthday foo'.split())
```

[58]: ('there', 'is', 'a', 'cat', 'licking', 'your', 'birthday', '<UNK>')

When it comes to ngram models the training boils down to counting up the ngrams from the training corpus.

```
[59]: print(lm.counts)
```

<NgramCounter with 2 ngram orders and 81 ngrams>

We can check how often individual unigrams occur.

```
[60]: lm.counts["licking"]
```

[60]: 1

```
[61]: lm.counts["birthday"]
```

[61]: 2

We can also check how often bigrams occur.

```
[62]: lm.counts[["might"]]["be"]
```

[62]: 1

The real purpose of training a language model is to have it score how probable words are in certain contexts. This being MLE, the model returns the item's relative frequency as its score.

```
[63]: lm.score("licking")
```

[63]: 0.023809523809523808

```
[64]: lm.score("birthday")
```

[64]: 0.047619047619047616

```
[65]: lm.score("be", ["might"])
```

```
[65]: 1.0
```

Items that are not seen during training are mapped to the vocabulary’s “unknown label” token. All unknown tokens have the same probability.

```
[66]: lm.score("<UNK>") == lm.score("foo")
```

```
[66]: True
```

```
[67]: lm.score("<UNK>") == lm.score("bar")
```

```
[67]: True
```

To avoid underflow when working with many small score values it makes sense to take their logarithm. For convenience this can be done with the `logscore` method.

```
[68]: lm.logscore("licking")
```

```
[68]: -5.392317422778761
```

```
[69]: lm.logscore("birthday")
```

```
[69]: -4.392317422778761
```

```
[ ]: lm.logscore("be", ["might"])
```

1.5 Is Software Natural?

Now that we know what a language model is, let’s return to software. The core of the naturalness hypothesis is, that software is similarly repetitive and predictable as natural language.

To determine how predictable a language is, a statistical language model, estimated carefully from a representative corpus, can be evaluated in terms of their *perplexity* with respect to the contents of a new document drawn from the same population. A good model can guess the contents of the new document with very high probability; i.e., it will not find the new document particularly surprising or perplexing.

The perplexity of a language model on a test set is the inverse probability of the test set, normalised by the number of words: $PP(W) = P(w_1 w_2 \dots w_N)^{-\frac{1}{N}}$

$$PP(W) = \sqrt[N]{\prod_{i=1}^N \frac{1}{P(w_i|w_{i-1})}}$$

Perplexity can also be seen as the weighted average branching factor of a language, i.e., the number of possible next words that can follow any word.

It is common to use the log-transformed variant of perplexity, called *cross entropy*:

$$H(s) = -\frac{1}{N} \log(P(a_1 \dots a_n))$$

NLTK of course offers a means to calculate the cross entropy. Let’s first pick a dataset.

```
[70]: import nltk
      from nltk.corpus import brown

      # Might be necessary the first time:
      # nltk.download('brown')
```

The Brown Corpus was the first million-word electronic corpus of English, created in 1961 at Brown University. This corpus contains text from 500 sources.

```
[71]: len(brown.words())
```

```
[71]: 1161192
```

In NLP it is common to apply various preprocessing steps before training a language model. We will keep it simple and just build a corpus of lower case versions of the words in the brown corpus.

```
[72]: brown = nltk.corpus.brown
      corpus = [[word.lower() for word in sent] for sent in brown.sents()]
```

```
[73]: corpus[0]
```

```
[73]: ['the',
      'fulton',
      'county',
      'grand',
      'jury',
      'said',
      'friday',
      'an',
      'investigation',
      'of',
      'atlanta's',
      'recent',
      'primary',
      'election',
      'produced',
      '\'',
      'no',
      'evidence',
      '"',
      'that',
      'any',
      'irregularities',
      'took',
      'place',
      '.']
```

Let's split the dataset into 95% training data, and 5 test data.

```
[74]: split = int(95*len(corpus)/100)
      train = corpus[:split]
      test  = corpus[split:]
```

Now we can build a language model as we did previously, using a maximum likelihood estimator.

```
[75]: n = 2
      train_data, padded_sents = padded_everygram_pipeline(n, train)
```

```
[76]: lm = MLE(n)
```

```
[77]: lm.fit(train_data, padded_sents)
```

To calculate the perplexity, we can use NLTK. The perplexity function in NLTK expects a list of n-grams as test set.

```
[78]: from nltk.lm.preprocessing import padded_everygrams
      from nltk.lm.preprocessing import flatten

      test_data = list(flatten(padded_everygrams(n, sent) for sent in test))
```

```
[79]: lm.perplexity(test_data)
```

```
[79]: inf
```

We can also calculate the log-transformed version of perplexity, the cross-entropy:

```
[80]: lm.entropy(test_data)
```

```
[80]: inf
```

Whoops, infinitely surprised?

This is a problem of data sparsity: Some n-grams may never occur in one corpus, but may in fact occur elsewhere. Consequently there may be some n-grams in the test data that are not in the training data.

Smoothing is a technique to handle cases where we have not seen the n-grams yet and still produce usable results with sufficient statistical rigor. There exist a variety of techniques for smoothing the estimates of a very large number of coefficients, some of which are larger than they should be and others smaller.

The simplest smoothing technique is Laplace smoothing, which adds 1 to the count for every n-gram. In practice, this is not a recommended approach, and there are more sophisticated smoothing techniques such as Good-Turing estimates, Jelinek-Mercer smoothing, Katz smoothing, Witten-Bell smoothing, Absolute discounting, Kneser-Ney smoothing, Modified Kneser Ney smoothing, and others.

```
[81]: from nltk.lm import Laplace
```

```
[82]: n = 3
train_data, padded_sents = padded_everygram_pipeline(n, train)
```

```
[83]: brown_model = Laplace(n)
brown_model.fit(train_data, padded_sents)
```

Let's first calculate the perplexity.

```
[84]: brown_model.perplexity(test_data)
```

```
[84]: 1427.0108783362869
```

...and now the cross entropy.

```
[85]: brown_model.entropy(test)
```

```
[85]: 15.523681195300634
```

Hindle et al. evaluated the cross entropy for different values of `n` on the Brown and the Gutenberg corpus. We will replicate this experiment, but to keep the computation time down we'll skip the Gutenberg corpus and only use small values for `n`, and no cross-validation. It is worth noting, however, that the perplexity of two language models is only *directly* comparable if they use identical vocabularies.

```
[86]: for n in range(1,5):
    train_data, padded_sents = padded_everygram_pipeline(n, train)
    brown_model = Laplace(n)
    brown_model.fit(train_data, padded_sents)
    entropy = brown_model.entropy(test_data)
    print(f"n = {n}: {entropy}")
```

```
n = 1: 13.246009780855404
n = 2: 10.455179613530257
n = 3: 10.478780617416895
n = 4: 10.515147986231895
```

To see whether software is similar, we need a corpus of source code. Unfortunately, NLTK does not provide this for us. We will thus use an existing corpus provided by others.

```
[ ]: # This may take a while so is commented out
     #!wget https://s3.amazonaws.com/code2seq/datasets/java-small.tar.gz
```

We will only need the lexemes rather than the full tokens, so let's define a helper function for this.

```
[87]: def tokenize(code):
    try:
        tokens = [token.value for token in javalang.tokenizer.tokenize(code)]
    except:
        # Parse errors may occur
        return []
    return tokens
```

We use this to create a training and test corpus, where a “sentence” is represented as the tokenized version of a Java source code file.

```
[88]: import tarfile

java_training = []
java_test = []
with tarfile.open("java-small.tar.gz", "r") as f:
    for tf in f.getmembers():
        if tf.isfile() and tf.name.startswith("java-small/train ing"):
            f2=f.extractfile(tf)
            content=f2.read()
            java_training.append(tokenize(content))
        elif tf.isfile() and tf.name.startswith("java-small/test"):
            f2=f.extractfile(tf)
            content=f2.read()
            java_test.append(tokenize(content))

len(java_training)
```

[88]: 89393

```
[89]: java_test_data = list(flatten(padded_everygrams(n, sent) for sent in java_test_
↪if sent))
```

Given this dataset, the steps to create a language model are identical to those for a natural language text.

```
[90]: for n in range(1,5):
        train_data, padded_sents = padded_everygram_pipeline(n, java_training)
        java_model = Laplace(n)
        java_model.fit(train_data, padded_sents)
        entropy = java_model.entropy(java_test_data)
        print(f"n = {n}: {entropy}")
```

```
n = 1: 17.193144615881664
n = 2: 15.054376176538463
n = 3: 13.477089769561248
n = 4: 12.435068706126893
```

1.6 Stopwords

In NLP it is common to remove stopwords before processing data. In our experiments we did not do this, and in particular there is the question what this means for source code: Intuitively, source code contains quite a substantial amount of syntactical overhead. The effects of this have been investigated in the following paper:

Rahman, M., Palani, D., & Rigby, P. C. (2019, May). Natural software revisited. In 2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE) (pp. 37-48). IEEE.

Lets also have a closer look at this. First we compare the language model on the Brown corpus with / without stopwords. We first build a 3-gram model with stopwords.

```
[91]: corpus = [[word for word in sent] for sent in brown.sents()]
      split = int(95*len(corpus)/100)
      train = corpus[:split]
      test = corpus[split:]

[92]: n = 3
      train_data, padded_sents = padded_everygram_pipeline(n, train)

      lm_with = Laplace(n)
      lm_with.fit(train_data, padded_sents)

[93]: test_data = list(flatten(padded_everygrams(n, sent) for sent in test))

[94]: lm_with.entropy(test)

[94]: 15.701397604007587
```

Now we build a pre-processed version of the corpus.

```
[95]: # nltk.download('stopwords')
      from nltk.corpus import stopwords
      stop_words = set(stopwords.words('english'))
      corpus_ns = [[word for word in sent if not word.lower() in stop_words] for sent_
        ↪in brown.sents()]

[96]: spl = int(95*len(corpus)/100)
      train_ns = corpus_ns[:spl]
      test_ns = corpus_ns[spl:]

      n = 3
      train_data, padded_sents = padded_everygram_pipeline(n, train_ns)

      lm_without = Laplace(n)
      lm_without.fit(train_data, padded_sents)

[97]: test_data = list(flatten(padded_everygrams(n, sent) for sent in test))

[98]: lm_without.entropy(test)

[98]: 15.708718590166072
```

Probably the effect is not large. However, let's now do this on source code.

```
[99]: n = 3
      train_data, padded_sents = padded_everygram_pipeline(n, java_training)
```

```
[100]: java_with = Laplace(n)
       java_with.fit(train_data, padded_sents)
```

```
[101]: java_with.entropy(java_test_data)
```

```
[101]: 13.477089769561248
```

Since our Java-corpus only contains the lexemes but no longer the token type information, we'll just re-build the corpus from scratch, but filter on separators.

```
[102]: def tokenize_without_stopwords(code):
       try:
           tokens = [token.value for token in javalang.tokenizer.tokenize(code) if
↳ not isinstance(token, javalang.tokenizer.Separator) ]
       except:
           return []
       return tokens
```

```
[103]: java_training = []
       java_test = []
       with tarfile.open("java-small.tar.gz", "r") as f:
           for tf in f.getmembers():
               if tf.isfile() and tf.name.startswith("java-small/training"):
                   f2 = f.extractfile(tf)
                   content = f2.read()
                   tokens = tokenize_without_stopwords(content)
                   if tokens:
                       java_training.append(tokens)
               elif tf.isfile() and tf.name.startswith("java-small/test"):
                   f2 = f.extractfile(tf)
                   content = f2.read()
                   tokens = tokenize_without_stopwords(content)
                   if tokens:
                       java_test.append(tokens)
```

```
[104]: n=3
       train_data, padded_sents = padded_everygram_pipeline(n, java_training)
```

```
[105]: java_without = Laplace(n)
       java_without.fit(train_data, padded_sents)
```

```
[106]: test_data = list(flatten(padded_everygrams(n, sent) for sent in java_test))
```

```
[107]: java_without.entropy(test_data)
```

```
[107]: 15.377227943201836
```

The entropy of Java without separator characters is higher than without – this shows that to a certain degree the repetitiveness of software is influenced by the syntactic overhead.

1.7 Code Completion

n-gram models can be used to generate text, and we start by doing this on a classical corpus of natural language text available at: <https://www.kaggle.com/datasets/kingburrito666/better-donald-trump-tweets?resource=download>

```
[108]: import pandas as pd
df = pd.read_csv('data/Donald-Tweets!.csv')
df.head()
```

```
[108]:      Date      Time      Tweet_Text \
0  16-11-11  15:26:37  Today we express our deepest gratitude to all ...
1  16-11-11  13:33:35  Busy day planned in New York. Will soon be mak...
2  16-11-11  11:14:20  Love the fact that the small groups of protest...
3  16-11-11   2:19:44  Just had a very open and successful presidenti...
4  16-11-11   2:10:46  A fantastic day in D.C. Met with President Oba...
```

```
      Type Media_Type  Hashtags      Tweet_Id \
0  text      photo  ThankAVet  7.970000e+17
1  text      NaN      NaN  7.970000e+17
2  text      NaN      NaN  7.970000e+17
3  text      NaN      NaN  7.970000e+17
4  text      NaN      NaN  7.970000e+17
```

```
      Tweet_Url \
0  https://twitter.com/realDonaldTrump/status/797...
1  https://twitter.com/realDonaldTrump/status/797...
2  https://twitter.com/realDonaldTrump/status/797...
3  https://twitter.com/realDonaldTrump/status/796...
4  https://twitter.com/realDonaldTrump/status/796...
```

```
      twt_favourites_IS_THIS_LIKE_QUESTION_MARK  Retweets  Unnamed: 10 \
0      127213      41112      NaN
1      141527      28654      NaN
2      183729      50039      NaN
3      214001      67010      NaN
4      178499      36688      NaN
```

```
      Unnamed: 11
0      NaN
1      NaN
2      NaN
3      NaN
4      NaN
```

We build the model as usual.

```
[109]: from nltk import word_tokenize
trump_corpus = list(df['Tweet_Text'].apply(word_tokenize))
```

```
[110]: # Preprocess the tokenized text for 3-grams language modelling
n = 3
train_data, padded_sents = padded_everygram_pipeline(n, trump_corpus)
```

```
[111]: trump_model = MLE(n)
trump_model.fit(train_data, padded_sents)
```

```
[112]: trump_model.generate(10)
```

```
[112]: ['election',
        'against',
        'Crooked',
        'Hillary',
        'Clinton',
        'said',
        'she',
        'should',
        'drop',
        'out']
```

Let's use a helper function to turn this into more readable sentences.

```
[113]: # Taken from https://www.kaggle.com/code/alvations/
       ↪n-gram-language-model-with-nltk/notebook

from nltk.tokenize.treebank import TreebankWordDetokenizer
detokenize = TreebankWordDetokenizer().detokenize

def generate_sent(model, num_words, random_seed=42):
    """
    :param model: An ngram language model from `nltk.lm.model`.
    :param num_words: Max no. of words to generate.
    :param random_seed: Seed value for random.
    """
    content = []
    for token in model.generate(num_words, random_seed=random_seed):
        if token == '<s>':
            continue
        if token == '</s>':
            break
        content.append(token)
    return detokenize(content)
```

```
[114]: generate_sent(trump_model, num_words=20, random_seed=0)
```

```
[114]: 'picks it up! Democrats numbers are down big in new Quinnipiac poll just
        released . Wow . Unbelievable crowd'
```

```
[115]: generate_sent(trump_model, num_words=20, random_seed=2)
```

```
[115]: 'via my Facebook page in St. Joseph, Michigan . Streaming live - join us today  
because of my constant'
```

```
[116]: generate_sent(trump_model, num_words=20, random_seed=21)
```

```
[116]: ': @realDonaldTrump More Veterans will vote for Trump!!!'
```

We can also provide a context for the prediction in terms of a sentence. The last (n-1) tokens of this sentence are used to find the most likely n-gram.

```
[117]: trump_model.generate(1, text_seed = "Democrats")
```

```
[117]: 'against'
```

Similarly, a simple approach to implement code completion is to build an n-gram model of source code, use the last (n-1) tokens as context, and look at the most likely n-gram.

Suppose we have typed `System.out.` and want to know what's next.

```
[118]: context = "System.out."
```

```
[119]: tokens = [token.value for token in list(javalang.tokenizer.tokenize(context))]
```

```
[120]: java_with.generate(1, text_seed = tokens)
```

```
[120]: 'println'
```

What about for-loops?

```
[121]: context = "for (int i = 0; i < model.size(); i"
```

```
[122]: tokens = [token.value for token in list(javalang.tokenizer.tokenize(context))]
```

```
[123]: java_with.generate(1, text_seed = tokens)
```

```
[123]: '++'
```